

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr

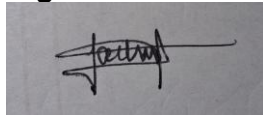
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5

9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I J a s h w a n t h . G . R certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Query Processing Pipeline

Inputs: a raw SQL string plus the system catalog (schema, indexes, statistics). Output: the formatted result set, or a descriptive error if the query turns out to be invalid at some stage. The pipeline is broken into four clearly separated stages - parsing, optimization, execution, and output - and each stage can fail on its own and should report an error specific to that stage.

```
FUNCTION ProcessQuery(sqlText):

    // ---- 1. Parsing ----
    tokens = Lexer.tokenize(sqlText)
    parseTree = Parser.buildParseTree(tokens)
    IF parseTree has syntax error:
        RETURN Error("Syntax error: " + parseTree.errorMessage)

    queryTree = Translator.toRelationalAlgebra(parseTree)
    IF queryTree references an unknown table/column:
        RETURN Error("Semantic error: unknown relation or attribute")

    // ---- 2. Optimization ----
    heuristicTree = ApplyHeuristicRules(queryTree)
    // push selections down, project early, reorder joins
    candidatePlans = GenerateCandidatePlans(heuristicTree)
    FOR EACH plan IN candidatePlans:
        plan.cost = EstimateCost(plan)    // uses catalog statistics
    bestPlan = the plan WITH MINIMUM cost IN candidatePlans

    // ---- 3. Execution ----
    executionPlan = CodeGenerator.compile(bestPlan)
    resultIterator = Executor.open(executionPlan)
    results = EMPTY LIST
    WHILE (tuple = resultIterator.next()) IS NOT NULL:
        APPEND tuple TO results
    resultIterator.close()

    // ---- 4. Result Output ----
    RETURN FormatResultSet(results)
```

Design notes

- Parsing itself happens in two steps: lexical/syntax checking against the SQL grammar, followed by semantic validation against the catalog (does the referenced table or column actually exist) - keeping them separate produces clearer error messages.
- The optimizer stage is kept deliberately generic here (heuristic rewrite -> candidate generation -> cost estimation -> pick the minimum) since Q2 works through the cost-comparison step in full detail.
- Execution relies on the standard iterator model (open/next/close) so operators can be pipelined without materializing every intermediate result.

Q2. Cost-Based Join Order Optimization

Inputs: relations $R_1..R_n$ to be joined along with their join predicates, plus the catalog statistics (tuple counts, block counts, index availability) needed to cost each access path. Output: whichever combination of join order and join method yields the lowest estimated total I/O cost. The classic System-R approach sidesteps trying every possible join order ($n!$ of them) by building the best plan for larger subsets out of the best plans already computed for smaller ones.

```
FUNCTION OptimizeJoinOrder(relations[R1..Rn], predicates):

    bestPlan = MAP()           // keyed by subset of relations

    // Base case: cheapest single-relation access path
    FOR EACH Ri IN relations:
        bestPlan[{Ri}] = CheapestAccessPath(Ri)    // scan vs index

    // Bottom-up dynamic programming over subsets
    FOR size = 2 TO n:
        FOR EACH subset S OF relations WITH |S| = size:
            bestPlan[S] = NULL
            FOR EACH split OF S INTO (S1, S2):
                IF a join predicate connects S1 and S2:
                    left = bestPlan[S1]
                    right = bestPlan[S2]
                    FOR EACH method IN {NestedLoop, SortMerge, Hash}:
                        cost = JoinCost(method, left, right)
                        IF bestPlan[S] IS NULL OR cost < bestPlan[S].cost:
                            bestPlan[S] = Plan(method, left, right, cost)

    RETURN bestPlan[ALL relations]

FUNCTION JoinCost(method, left, right):
    IF method == NestedLoop:
        RETURN left.blocks + CEIL(left.blocks/(nB-2)) * right.blocks
    IF method == SortMerge:
        RETURN SortCost(left) + SortCost(right) + (left.blocks+right.blocks)
    IF method == Hash:
        RETURN 2*(left.blocks+right.blocks) + (left.blocks+right.blocks)
```

Design notes

- The dynamic-programming table `bestPlan[]` guarantees that each subset's optimal sub-plan is reused rather than recomputed - this is exactly what keeps the search from growing exponentially with the number of relations.
- Only subset splits linked by an actual join predicate are considered; a split with no predicate would force a Cartesian product, which is always dominated by a plan that joins connected relations first.
- `JoinCost()` plugs in the same cost formulas used for individual join methods (nested-loop, sort-merge, hash), so the optimizer compares methods on equal footing at every subset size rather than committing to one method upfront.

Q3. Nested Loop Join and Complexity Analysis

Inputs: outer relation R (bR blocks, nR tuples), inner relation S (bS blocks, nS tuples), a join condition, and nB available buffer pages (two of which are reserved for an output block and an input block of S, leaving nB-2 for chunks of R).

```
FUNCTION NestedLoopJoin(R, S, condition, nB):  
    // R = outer relation (bR blocks), S = inner relation (bS blocks)  
    // nB = number of buffer pages available  
  
    result = EMPTY LIST  
  
    FOR EACH chunk OF (nB - 2) blocks of R:           // outer loop  
        LOAD chunk into memory buffer  
        FOR EACH block Sblk OF S:                     // inner loop  
            LOAD Sblk into memory buffer  
            FOR EACH tuple r IN chunk:  
                FOR EACH tuple s IN Sblk:  
                    IF condition(r, s) IS TRUE:  
                        ADD (r, s) TO result  
  
    RETURN result
```

Time complexity analysis

Block (I/O) cost: R is read exactly once (bR block transfers), and for each of the $\lceil bR/(nB-2) \rceil$ chunks, the entire inner relation S is re-scanned, giving a total I/O cost of:

- Cost = $bR + \lceil bR/(nB-2) \rceil \times bS$ block accesses.

Tuple-comparison cost: in the worst case (no index, no sorting) every tuple of R is compared against every tuple of S, so the comparison count is $nR \times nS$ - which is why a plain nested-loop join is described as $O(nR \times nS)$: quadratic in the input sizes. The block-level formula above just reflects how much of that comparison work gets amortized per disk read as more buffer pages (nB) become available. As nB grows, larger chunks of R stay in memory per pass, S gets re-scanned fewer times, and the I/O cost - though never the tuple-comparison cost itself - improves.

Q4. Deadlock Detection via Wait-For Graph

Inputs: the current Wait-For Graph (WFG), where a directed edge $T_i \rightarrow T_j$ means T_i is blocked waiting on a lock held by T_j . Output: TRUE/FALSE for whether a deadlock currently exists, and if it does, a victim to abort. A deadlock exists exactly when the graph contains a cycle, so the problem reduces to standard graph cycle detection via depth-first search with a recursion-stack set.

```

FUNCTION DetectDeadlock(WFG):
    visited = EMPTY SET
    onStack = EMPTY SET

    FOR EACH node T IN WFG.vertices:
        IF T NOT IN visited:
            IF HasCycle(T, visited, onStack, WFG):
                RETURN TRUE // a wait-for cycle exists

    RETURN FALSE

FUNCTION HasCycle(T, visited, onStack, WFG):
    ADD T TO visited
    ADD T TO onStack

    FOR EACH neighbor N IN WFG.edges[T]: // T is waiting for N
        IF N IN onStack:
            RETURN TRUE // back-edge -> cycle
        IF N NOT IN visited:
            IF HasCycle(N, visited, onStack, WFG):
                RETURN TRUE

    REMOVE T FROM onStack
    RETURN FALSE

FUNCTION ResolveDeadlock(cycle):
    victim = SELECT transaction IN cycle WITH least work done
    ABORT victim
    RELEASE all locks held BY victim
    WAKE UP transactions waiting on those locks

```

Design notes

- onStack tracks the nodes on the current DFS recursion path; hitting an edge into a node still on that path (a back-edge) is exactly what proves a cycle, with no need to store or compare full paths.
- visited (kept separate from onStack) ensures each node's outgoing edges are explored only once overall, which keeps the algorithm $O(V + E)$ - linear in the number of transactions and wait edges - instead of re-exploring shared sub-paths.
- This detection routine typically gets re-run periodically (or on every new wait-edge insertion) by the lock manager; ResolveDeadlock() then picks a victim (say, least work done, or the youngest transaction) - much the way Q6's Wait-Die scheme picks one by timestamp instead.

Q5. Basic Two-Phase Locking Protocol

Inputs: a transaction T_i 's sequence of read/write operations, and the shared lock table with a per-item wait queue.

Output: the transaction runs to completion, acquiring all its locks in a single growing phase and releasing them all together in a single shrinking phase - the defining 2PL rule being that no new lock may be requested once the first lock has been released.

```

FUNCTION RequestLock(Ti, item, mode):
  IF item has no lock holder:
    GRANT lock(Ti, item, mode)
    RETURN GRANTED
  ELSE IF Compatible(currentLocks(item), mode):
    GRANT lock(Ti, item, mode)
    RETURN GRANTED
  ELSE:
    ADD Ti TO item.waitQueue
    BLOCK Ti UNTIL woken and lock becomes compatible
    RETURN GRANTED

FUNCTION Transaction_Execute(Ti, operations):
  phase = GROWING

  FOR EACH op IN operations:
    IF op == Read(item):
      RequestLock(Ti, item, SHARED)
    IF op == Write(item):
      RequestLock(Ti, item, EXCLUSIVE)
    PERFORM op
  // 2PL rule: every lock above is acquired before any lock
  // is released below - growing and shrinking never overlap
  phase = SHRINKING

  ON Commit(Ti) OR Abort(Ti):
    FOR EACH lock HELD BY Ti:
      RELEASE lock
      WAKE UP transactions IN lock.waitQueue

```

Design notes

- Compatible(mode1, mode2) is the usual lock-compatibility check: two SHARED locks are compatible with each other, but any EXCLUSIVE request conflicts with whatever lock already sits on that item.
- Blocking a transaction on item.waitQueue rather than aborting it is what makes this Basic (as opposed to Conservative or Strict) 2PL - it guarantees conflict-serializability but not, on its own, freedom from deadlock (that needs the detection routine from Q4, or a prevention scheme like Q6).
- Releasing all locks together only at commit/abort, and waking every waiting transaction at that point, keeps the two-phase property intact and gives the lock manager its next chance to grant queued requests.

Q6. Timestamp Ordering with Thomas' Write Rule

Inputs: transaction T_i with a fixed start timestamp $TS(T_i)$; every data item carries a ReadTS (timestamp of the youngest transaction to read it) and a WriteTS (timestamp of the youngest transaction to write it). Output: each read/write either executes, gets silently ignored (Thomas' Write Rule only), or aborts T_i - it never blocks, which is the key advantage timestamp ordering has over lock-based schemes.

```

FUNCTION TSRead(Ti, item):
  IF TS(Ti) < WriteTS(item):
    ABORT Ti          // item was overwritten by a younger Tx
  ELSE:
    EXECUTE Read(item)
    ReadTS(item) = MAX(ReadTS(item), TS(Ti))

FUNCTION TSWrite(Ti, item):
  IF TS(Ti) < ReadTS(item):
    ABORT Ti          // a younger transaction already read
                      // the value this write would overwrite
  ELSE IF TS(Ti) < WriteTS(item):
    // ---- Thomas' Write Rule ----
    IGNORE this Write(item)
    // a younger transaction has already written a later value,
    // so this obsolete write can be safely skipped (no abort)
  ELSE:
    EXECUTE Write(item)
    WriteTS(item) = TS(Ti)

```

Design notes

- Ordinary timestamp ordering would abort T_i in the ELSE IF branch too ($TS(T_i) < WriteTS(item)$); Thomas' Write Rule relaxes precisely that case - since a younger transaction has already written a later value, this write could never be seen by anyone anyway, so silently skipping it preserves a serializable outcome without paying for an abort and restart.
- The rule only ever relaxes write-write conflicts this way - a write that would violate the read timestamp ($TS(T_i) < ReadTS(item)$) still has to abort, since skipping it could let a later reader miss a value it should have seen.

Q7. ARIES Recovery (Analysis, Redo, Undo)

Inputs: the write-ahead log (carrying LSNs, prevLSN chains, and periodic checkpoints), plus the buffer pool's dirty-page and transaction tables as they stood at crash time (assumed lost and needing to be rebuilt). Output: a database state as if every committed transaction's effects are durably applied and every uncommitted (loser) transaction's effects are fully removed - ARIES gets there with three sequential passes over the log.


```

FUNCTION ARIESRecovery(log):

    // ---- Phase 1: Analysis ----
    dirtyPageTable = EMPTY MAP
    transactionTable = EMPTY MAP
    FOR EACH record IN log FROM lastCheckpoint TO end:
        IF record == START(Ti):
            transactionTable[Ti] = ACTIVE
        IF record == UPDATE(Ti, page, ...):
            IF page NOT IN dirtyPageTable:
                dirtyPageTable[page] = record.LSN
                transactionTable[Ti].lastLSN = record.LSN
        IF record == COMMIT(Ti) OR record == END(Ti):
            REMOVE Ti FROM transactionTable

    // ---- Phase 2: Redo ----
    redoStart = MIN(recLSN) IN dirtyPageTable
    FOR EACH record IN log FROM redoStart TO end:
        IF record == UPDATE AND page.LSN < record.LSN:
            REAPPLY record.change TO page
            page.LSN = record.LSN

    // ---- Phase 3: Undo ----
    losers = transactions IN transactionTable STILL ACTIVE
    toUndo = MAX-HEAP of losers' lastLSN
    WHILE toUndo NOT EMPTY:
        rec = log[toUndo.popMax()]
        IF rec == UPDATE:
            WRITE rec.oldValue TO rec.item
            APPEND CompensationLogRecord(rec) TO log
            toUndo.push(rec.prevLSN)
        IF rec == START of a loser:
            APPEND End(Ti) TO log

    RETURN "database restored to a consistent state"

```

Design notes

- Analysis rebuilds exactly the in-memory tables lost in the crash (which pages were dirty and from what LSN, which transactions were still active) by scanning forward from the last checkpoint - it performs no actual recovery work yet.
- Redo starts from the earliest recLSN found in the dirty-page table (not from the very start of the log) and reapplies every logged change whose page.LSN shows it never reached disk - this repeating-history approach is what makes ARIES efficient rather than redoing the entire log.
- Undo rolls back only the loser transactions, writing a Compensation Log Record for every change it undoes, so that if the system crashes again mid-recovery, those undo actions are never repeated.

Q8. Shadow Paging Recovery

Inputs: a current page table (the last committed, consistent version of the database) and, per active transaction, a shadow page table that starts out as an exact copy of the current one. Output: on commit, the shadow table atomically

becomes the new current table; on rollback or crash, the shadow table is simply discarded and the original current table - untouched the whole time - remains valid, so no UNDO log is ever needed.

```
FUNCTION ShadowWrite(Ti, pageId, newData):
    IF pageId NOT IN Ti.copiedPages:
        newFrame = ALLOCATE a free disk block
        COPY currentPageTable[pageId] contents INTO newFrame
        Ti.shadowPageTable[pageId] = newFrame
        ADD pageId TO Ti.copiedPages
    WRITE newData TO Ti.shadowPageTable[pageId]    // shadow copy only

FUNCTION Commit(Ti):
    FOR EACH pageId IN Ti.copiedPages:
        currentPageTable[pageId] = Ti.shadowPageTable[pageId]
    ATOMICALLY SWAP the disk pointer to currentPageTable
    RECLAIM the old (pre-commit) page-table version's disk space

FUNCTION Rollback(Ti):
    DISCARD Ti.shadowPageTable and Ti.copiedPages
    // currentPageTable was never modified, so nothing to undo
    RETURN "transaction rolled back, database unchanged"
```

Design notes

- A page is copy-on-write: it's duplicated into the transaction's shadow table only the first time that transaction writes to it, so unmodified pages stay shared (pointed to by both tables) rather than being duplicated.
- Commit boils down to a single atomic pointer swap of the page-table root, which is also why shadow paging needs no REDO - the old version stays valid disk data until the swap succeeds, and the new version only becomes visible once every page has been safely written.
- The main drawback (not shown in the pseudocode but worth noting) is that committed pages end up scattered across disk over time, and the space held by now-unreferenced old pages eventually needs reclaiming - which is why shadow paging sees less use in high-throughput systems than log-based recovery such as ARIES.

Q9. Immediate Update Recovery (UNDO/REDO)

Inputs: the transaction log recording $\langle Ti \text{ start} \rangle$, $\langle Ti, \text{item}, \text{oldValue}, \text{newValue} \rangle$ update records, and $\langle Ti \text{ commit} \rangle$ records, written under a protocol where a transaction's writes may reach disk before it commits. Output: every committed transaction's changes are guaranteed to be present (REDO), and every uncommitted transaction's changes are fully reversed (UNDO).

```

FUNCTION ImmediateUpdateRecovery(log):
    undoList = EMPTY SET
    redoList = EMPTY SET

    // ---- Classify transactions (forward scan) ----
    FOR EACH record IN log:
        IF record == START(Ti):
            ADD Ti TO undoList
        IF record == COMMIT(Ti):
            REMOVE Ti FROM undoList
            ADD Ti TO redoList

    // ---- UNDO pass: scan log backward ----
    FOR EACH record IN log (reverse order):
        IF record.Tid IN undoList AND record == UPDATE:
            WRITE record.oldValue TO record.item

    // ---- REDO pass: scan log forward ----
    FOR EACH record IN log (forward order):
        IF record.Tid IN redoList AND record == UPDATE:
            WRITE record.newValue TO record.item

    RETURN "recovery complete"

```

Design notes

- Transactions are classified in a single forward scan: starting a transaction tentatively marks it for UNDO, and encountering its commit record promotes it to REDO instead - so by the end of the scan, every transaction sits in exactly one list.
- UNDO has to run in reverse chronological order (undoing later writes first, in case the same item was written more than once), while REDO has to run forward - so the two passes deliberately scan the log in opposite directions.
- This is the general (UNDO/REDO) counterpart to Deferred Update's NO-UNDO/REDO approach from the earlier assessment - the difference comes entirely from Immediate Update letting uncommitted writes reach disk, which is exactly what makes the UNDO pass necessary.

Q10. Database Connectivity Manager with Connection Pooling

Inputs: a SQL statement with its bound parameters, plus a bounded pool of reusable database connections (opening a fresh connection for every query is expensive). Output: the query result on success, or a clean rollback with a reported error on failure - with the connection always returned to the pool afterward regardless of outcome.

```

CLASS ConnectionPool:
    idleConnections = EMPTY QUEUE
    activeConnections = EMPTY SET
    maxSize = N

FUNCTION GetConnection():
    IF idleConnections NOT EMPTY:
        conn = idleConnections.dequeue()
    ELSE IF SIZE(activeConnections) < maxSize:
        conn = OpenNewConnection(dbUrl, credentials)
    ELSE:
        WAIT UNTIL a connection is released
        conn = idleConnections.dequeue()
    ADD conn TO activeConnections
    RETURN conn

FUNCTION ExecuteQuery(sql, params):
    conn = GetConnection()
    TRY:
        stmt = conn.prepareStatement(sql)
        BIND params TO stmt
        conn.beginTransaction()
        result = stmt.executeQuery()
        conn.commit()
        RETURN result
    CATCH SQLException AS e:
        conn.rollback()
        LOG "Query failed: " + e.message
        RETURN Error(e)
    FINALLY:
        ReleaseConnection(conn)

FUNCTION ReleaseConnection(conn):
    REMOVE conn FROM activeConnections
    IF conn.isValid():
        ADD conn TO idleConnections
    ELSE:
        CLOSE conn
    NOTIFY one thread WAITING in GetConnection()

```

Design notes

- The pool caps the number of simultaneously open connections at `maxSize`; once that limit is reached, new requests wait for a release instead of opening unbounded connections, which is what protects the database server from being overwhelmed.
- TRY / CATCH / FINALLY mirrors the try-with-resources pattern from the JDBC answer in the earlier assessment: commit only happens along the success path, any `SQLException` triggers an explicit rollback, and the FINALLY block guarantees the connection is always released - even when an exception is thrown - so a failed query can never leak a connection out of the pool.

- `conn.isValid()` is checked before a connection goes back into the idle queue, so a connection broken by the failure (say, a dropped network socket) is closed and discarded rather than handed to the next caller.